

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Estructura de Datos

Sección: A

Docente: Mario Moisés Ramírez Tobar



“Reporte TAD’s Proyecto 1 Fase 2”

Estudiante:

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 04 de mayo de

2026

TAD's de Estructuras - Proyecto Fase 2

1. TAD Producto (Actualizado)

Estructura que representa un producto del supermercado. Ahora incluye estado de transferencia.

Dominio:

text

(nombre, codigoBarras, categoria, fechaVencimiento, marca, precio, stock,
sucursal_id, estado, origen_traslado, sucursal_actual)

Atributos:

Atributo	Tipo	Descripción
nombre	string	Nombre del producto
codigo_barras	string	Identificador único (mínimo 10 dígitos)
categoria	string	Categoría del producto
fecha_vencimiento	string	Formato YYYY-MM-DD
marca	string	Marca del producto
precio	double	Precio en quetzales
stock	int	Cantidad disponible
sucursal_id	string	ID de la sucursal donde está
estado	string	"disponible", "en_transito", "en_cola_ingreso", "en_cola_traspaso", "en_cola_salida", "despachado"
origen_traslado	string	Sucursal que envió el producto
sucursal_actual	string	Sucursal donde se encuentra actualmente

Estructura	Propósito	Clave de ordenación
------------	-----------	---------------------

Lista Enlazada Almacenamiento básico + comparativa (ninguna / nombre)

Árbol AVL Búsqueda por nombrenombre

Árbol B Búsqueda por rango de fechas fecha_vencimiento

Árbol B+ Búsqueda por categoría categoria

Tabla Hash Búsqueda por código de barras codigo_barras

Pila Rollback (deshacer eliminaciones) N/A

Cola Simulación de flujo de productos N/A (FIFO)

Grafo Red de sucursales y rutas óptimas tiempo / costo

Operaciones:

Operación	Prototipo	Complejidad
Constructor por defecto	Producto()	O(1)
Constructor parametrizado	Producto(sucursal_id, nombre, codigo_barras, categoria, fecha_vencimiento, marca, precio, stock)	O(1)
str	string __str__()	O(1)

2. TAD NodoListaEnlazada

(Similar a Fase 1 - se mantiene)

Elemento para la construcción de listas dinámicas.

Dominio:

text

(valor, siguiente)

valor: Elemento de tipo genérico T (Producto)

siguiente: Puntero a otro NodoListaEnlazada<T> o None

Operaciones:

Operación	Prototipo	Complejidad
Constructor	NodoListaEnlazada(const T& valor)	O(1)

get_valor	T get_valor()	O(1)
get_siguiente	Nodo* get_siguiente()	O(1)
set_siguiente	void set_siguiente(Nodo* siguiente)	O(1)

3. TAD ListaEnlazada

(Actualizado de Fase 1)

Estructura lineal dinámica. Se utiliza para búsqueda secuencial (comparativa) y almacenamiento de productos.

Dominio:

text

(head, size)

Operación	Prototipo	Complejidad
Constructor	ListaEnlazada()	O(1)
insertar_al_final	void insertar_al_final(const T& valor)	O(n) - recorre hasta el final
insertar_ordenado_por_nombre	void insertar_ordenado_por_nombre(const Producto& p)	O(n) - busca posición de inserción
eliminar_por_codigo	bool eliminar_por_codigo(string codigo)	O(n)
get_size	int get_size()	O(1)
get_head	Nodo* get_head()	O(1)

4. TAD NodoArbolAVL

(Similar a Fase 1 - se mantiene)

Elemento para la construcción del árbol AVL.

Dominio:

text

(valor, izq, der, fe)

valor: Producto

izq: NodoArbolAVL* - Hijo izquierdo

der: NodoArbolAVL* - Hijo derecho

fe: int - Factor de equilibrio

Operaciones:

Operación	Prototipo	Complejidad
Constructor	NodoArbolAVL(const Producto& p)	O(1)
get_valor, get_izq, get_der, get_fe	Getters	O(1)
set_valor, set_izq, set_der, set_fe	Setters	O(1)

5. TAD ArbolAVL

(Similar a Fase 1 - se mantiene)

Árbol binario de búsqueda auto-balanceado. Ordenado por nombre del producto.

Dominio:

text

(raiz)

Operaciones:

Operación	Prototipo	Complejidad
insertar	bool insertar(const Producto& p)	O(log n)
buscar	Producto* buscar(const string& nombre)	O(log n)
eliminar	bool eliminar(const string& nombre)	O(log n)
_rotacion_LL, _DD, _ID, _DI	Nodo* rotacion(Nodo* nodo)	O(1)
_balancear	Nodo* _balancear(Nodo* nodo)	O(1)
is_empty	bool is_empty()	O(1)

get_raiz Nodo* get_raiz() O(1)

6. TAD NodoArbolB

(Similar a Fase 1 - se mantiene)

Página del árbol B. Puede contener múltiples llaves y punteros a hijos.

Dominio:

text

(llaves[MAX_LLAVES], hijos[MAX_HIJOS], num_llaves, es_hoja)

Constantes:

text

ORDEN_B = 2

MAX_LLAVES = 4

MAX_HIJOS = 5

Operaciones:

Operación	Prototipo	Complejidad
Constructor	NodoArbolB(bool hoja = True)	O(1)
esta_lleno	bool esta_lleno()	O(1)
getters/setters básicos	varios	O(1)

7. TAD ArbolB

(Similar a Fase 1 - se mantiene)

Árbol B de orden 2. Ordenado por fecha de vencimiento. Ideal para búsquedas por rango.

Dominio:

text

(raiz)

Operaciones:

Operación	Prototipo	Complejidad
insertar	bool insertar(const Producto& p)	$O(\log n)$
buscar	Producto* buscar(const string& fecha)	$O(\log n)$
buscar_por_rango	list buscar_por_rango(desde, hasta)	$O(\log n + k)$
eliminar	bool eliminar(const string& fecha)	$O(\log n)$
_split	void _split(padre, i, hijo_lleno)	$O(1)$
_fusionar	void _fusionar(nodo, idx)	$O(1)$
_prestar	void _prestar_de_anterior/siguiente(...)	$O(1)$

8. TAD NodoArbolBPlus

(NUEVO para Fase 2)

Nodo del árbol B+. Similar al nodo B, pero con puntero adicional para enlazar hojas.

Dominio:

text

(llaves[MAX_LLAVES_BP], hijos[MAX_HIJOS_BP], num_llaves, es_hoja, siguiente)

siguiente: NodoArbolBPlus* - Puntero a la siguiente hoja (solo relevante en hojas)

Constantes:

text

ORDEN_BP = 2

MAX_LLAVES_BP = 4

MAX_HIJOS_BP = 5

Operaciones (adicionales a NodoArbolB):

Operación	Prototipo	Complejidad
-----------	-----------	-------------

get_siguiente	NodoArbolBPlus* get_siguiente()	O(1)
set_siguiente	void set_siguiente(NodoArbolBPlus* sig)	O(1)

9. TAD ArbolBPlus

(NUEVO para Fase 2)

Árbol B+ de orden 2. Todas las llaves están en las hojas, y las hojas están enlazadas. Ordenado por categoría del producto.

Dominio:

text

(raíz)

Operaciones:

Operación	Prototipo	Complejidad
insertar	void insertar(const Producto& p)	O(log n)
buscar	Producto* buscar(const string& categoria)	O(log n)
buscar_por_categoria	list<Producto> buscar_por_categoria(const string& categoria)	O(log n + k)
eliminar	bool eliminar(categoria, codigo)	O(log n)
_split	void _split(padre, i, hijo_lleno)	O(1)
_fusionar_hojas	void _fusionar_hojas(padre, idx)	O(1)
_fusionar_internos	void _fusionar_internos(padre, idx)	O(1)
_prestar_hoja_anterior/siguiente	void _prestar_...(...)	O(1)

Ventaja clave: Las hojas enlazadas permiten recorrido eficiente por categoría sin necesidad de volver a bajar desde la raíz.

10. TAD TablaHash

(Similar a Fase 1 - se mantiene)

Estructura que permite búsquedas por clave única con tiempo promedio $O(1)$. Implementa encadenamiento con ListaEnlazada.

Dominio:

text

(tabla[TAMANIO])

Constantes:

text

TAMANIO = 101 (número primo para mejor distribución)

Operaciones:

Operación	Prototipo	Complejidad
_funcion_hash	int _funcion_hash(string llave)	$O(k)$ - k =longitud llave
insertar	void insertar(const Producto& p)	$O(1)$ promedio
buscar	Producto* buscar(string codigo)	$O(1)$ promedio
eliminar	void eliminar(string codigo)	$O(1)$ promedio
get_cubeta	ListaEnlazada get_cubeta(int i)	$O(1)$
get_tamanio	int get_tamanio()	$O(1)$

Función hash (djb2):

text

hash = 5381

for c in llave:

hash = ((hash << 5) + hash) + ord(c)

return hash % TAMANIO

11. TAD Pila (para Rollback)

(Similar a Fase 1 - se mantiene)

Estructura LIFO utilizada para el rollback (deshacer eliminaciones).

Dominio:

text

(items)

Operaciones:

Operación	Prototipo	Complejidad
push	void push(const T& valor)	O(1)
pop	T pop()	O(1)
esta_vacia	bool esta_vacia()	O(1)

12. TAD Cola

(NUEVO para Fase 2)

Estructura FIFO utilizada para simular el flujo de productos entre sucursales. Implementa colas de ingreso, traspaso y salida.

Dominio:

text

(items)

items: list (se usa como cola, solo operaciones FIFO)

Operación	Prototipo	Complejidad
Constructor	Cola()	O(1)
enqueue	void enqueue(const T& valor)	O(1) - agrega al final

dequeue	T dequeue()	$O(1)^*$ - remueve del inicio
esta_vacia	bool esta_vacia()	$O(1)$

****Nota:** En implementación con lista de Python, pop(0) es $O(n)$; se puede mejorar con collections.deque.*

Uso específico por sucursal:

Cola Propósito

cola_ingreso Productos recién recibidos (destino final)

cola_traspaso Productos en preparación para reenviar (sucursal intermedia)

cola_salida Productos listos para ser enviados

13. TAD Grafo

(NUEVO para Fase 2)

Grafo no dirigido ponderado que representa la red de sucursales. Implementa los algoritmos de Dijkstra y Floyd-Warshall para cálculo de rutas óptimas.

Dominio:

text

(adyacencia, vertices)

adyacencia: Diccionario {sucursal: [(vecino, tiempo, costo), ...]}

vertices: Lista de nombres de sucursales (para Floyd-Warshall)

Operaciones:

Operación	Prototipo	Complejidad
agregar_sucursal	void agregar_sucursal(string nombre)	$O(1)$

agregar_camino	void agregar_camino(origen, destino, tiempo, costo)	$O(1)$
dijkstra	(list camino, float costo) dijkstra(inicio, fin, criterio)	$O((V+E) \log V)$
floyd	(matriz M, matriz T) floyd()	$O(V^3)$
obtener_ruta	list obtener_ruta(origen, destino, criterio)	$O((V+E) \log V)$
obtener_ruta_floyd	list obtener_ruta_floyd(origen, destino)	$O(V^3)$ precálculo + $O(1)$ consulta
imprimir_grafo	void imprimir_grafo()	$O(V+E)$
generar_dot	string generar_dot(camino_resaltado)	$O(V+E)$

Algoritmo Dijkstra (con cola de prioridad):

Inicializar distancias = ∞ , distancia[inicio] = 0

Usar heap para extraer el nodo con menor distancia

Relajar aristas: si nueva_dist < D[w], actualizar

Retornar camino y costo total

Algoritmo Floyd-Warshall:

Inicializar $M[i][j]$ = peso si existe arista, 0 si $i=j$, ∞ en otro caso

Para cada k, i, j: si $M[i][k] + M[k][j] < M[i][j]$, actualizar

$T[i][j]$ guarda el vértice intermedio k

Criterios de optimización:

"tiempo": minimiza el tiempo total de traslado

"costo": minimiza el costo total de traslado

14. TAD Sucursal

(NUEVO para Fase 2)

Representa una sucursal del supermercado con su catálogo, tiempos de procesamiento y colas.

Dominio:

text

(id, nombre, ubicacion, t_ingreso, t_traspaso, t_despacho,
catalogo, cola_ingreso, cola_traspaso, cola_salida)

Atributos:

Atributo	Tipo	Descripción
id	string	Identificador único de la sucursal
nombre	string	Nombre de la sucursal
ubicacion	string	Dirección o ubicación
t_ingreso	float	Tiempo en segundos para procesar ingreso
t_traspaso	float	Tiempo en segundos para preparar traspaso
t_despacho	float	Tiempo entre envíos (intervalo)
catalogo	Catalogo	Catálogo de productos de la sucursal
cola_ingreso	Cola	Cola de productos recibidos
cola_traspaso	Cola	Cola de productos en preparación
cola_salida	Cola	Cola de productos listos para enviar

Operaciones:

Operación	Prototipo	Complejidad
Constructor	Sucursal(id, nombre, ubicacion, t_ingreso, t_traspaso, t_despacho)	O(1)

15. TAD Catalogo

(NUEVO para Fase 2 - Evolución del Fase 1)

Clase coordinadora que mantiene todas las estructuras sincronizadas para una sucursal.
Implementa operaciones atómicas con rollback en caso de fallo.

Dominio:

text

(sucursal_id, lista, lista_ordenada, avl, arbol_b, arbol_b_plus, hash, historial_eliminados)

Atributos:

Atributo	Tipo	Propósito
sucursal_id	string	ID de la sucursal dueña
lista	ListaEnlazada	Lista sin orden (referencia)
lista_ordenada	ListaEnlazada	Lista ordenada por nombre
avl	ArbolAVL	Búsqueda por nombre
arbol_b	ArbolB	Búsqueda por rango de fechas
arbol_b_plus	ArbolBPlus	Búsqueda por categoría
hash	TablaHash	Búsqueda por código de barras
historial_eliminados	Pila	Rollback de eliminaciones

Operaciones:

Operación	Prototipo	Complejidad
agregar_producto	bool agregar_producto(Producto p)	$O(\log n)$ promedio
eliminar_producto	void eliminar_producto(string codigo)	$O(\log n)$ promedio
rollback	void rollback()	$O(\log n)$ promedio
buscar_por_codigo	Producto* buscar_por_codigo(string codigo)	$O(1)$ promedio
buscar_por_nombre	Producto* buscar_por_nombre(string nombre)	$O(\log n)$
buscar_por_categoria	list buscar_por_categoria(string categoria)	$O(\log n + k)$
buscar_por_rango	list buscar_por_rango(desde, hasta)	$O(\log n + k)$
get_avl / get_arbol_b / ...	getters	$O(1)$

Operación atómica de inserción:

text

1. Insertar en AVL → si falla, abortar
2. Insertar en lista → si falla, rollback AVL
3. Insertar en lista_ordenada → si falla, rollback AVL + lista
4. Insertar en árbol B → si falla, rollback estructuras anteriores
5. Insertar en árbol B+ → si falla, rollback estructuras anteriores
6. Insertar en Hash → si falla, rollback todas las anteriores
7. Retornar True (éxito)

16. TAD Logger

(NUEVO para Fase 2)

Sistema de logging para registrar eventos, errores y duplicados durante la carga de archivos CSV.

Dominio:

text

(archivo, conteo_errores_formato, conteo_errores_datos, conteo_duplicados)

Operaciones:

Operación	Prototipo	Complejidad
Constructor	Logger(nombre_archivo)	O(1)
info	void info(string mensaje)	O(1)
error	void error(string mensaje)	O(1)
error_formato	void error_formato(string mensaje)	O(1)

17. TAD CSVLoader

(NUEVO para Fase 2)

Carga de datos desde archivos CSV con validaciones robustas.

17.1 CSVLoader (Productos)

Operaciones:

Operación	Prototipo	Complejidad
cargar_archivo	bool cargar_archivo(ruta, callback)	O(n) - recorre líneas

Validaciones:

Número de campos (8 esperados)

Código de barras mínimo 10 dígitos

Precio > 0

Stock >= 0

Formato de fecha YYYY-MM-DD

17.2 CSVLoaderSucursales

Operaciones:

Operación	Prototipo	Complejidad
cargar_archivo	bool cargar_archivo(ruta, callback)	O(n)

Validaciones:

6 campos esperados

Tiempos no negativos

17.3 CSVLoaderConexiones

Operaciones:

Operación	Prototipo	Complejidad
cargar_archivo	bool cargar_archivo(ruta, callback)	O(n)

Validaciones:

4 campos esperados

Origen/destino no vacíos

Tiempo y costo ≥ 0

18. TAD Benchmark

(NUEVO para Fase 2)

Medición de rendimiento de operaciones con alta precisión (microsegundos).

Dominio:

text

(_inicio, _nombre_tarea)

Operaciones:

Operación	Prototipo	Complejidad
iniciar	void iniciar(string tarea)	O(1)
finalizar	void finalizar()	O(1)

Uso:

python

```
bench = Benchmark()
```

```
bench.iniciar("Búsqueda AVL")
```

```
producto = catalogo.buscar_por_nombre("Leche")
```

```
bench.finalizar()
```

Output: [BENCHMARK] Búsqueda AVL: 14 μ s

19. TAD ReportesGraphviz

(NUEVO para Fase 2)

Generación de visualizaciones gráficas de todas las estructuras de datos utilizando Graphviz.

Dominio:

text

(carpeta, carpeta_extra)

Operaciones:

Operación	Prototipo	Complejidad
Constructor	ReportesGraphviz(carpeta_salida, carpeta_extra)	$O(1)$
grafo_sucursales	string grafo_sucursales(Grafo, camino_resaltado)	$O(V+E)$
tabla_hash	string tabla_hash(TablaHash)	$O(n)$
arbol_avl	string arbol_avl(ArbolAVL)	$O(n)$
arbol_b	string arbol_b(ArbolB)	$O(n)$
arbol_b_plus	string arbol_b_plus(ArbolBPlus)	$O(n)$

Formatos generados: .dot, .png, .svg

20. TAD FlaskApp (Interfaz Web)

(NUEVO para Fase 2)

Aplicación web que expone todas las funcionalidades del sistema mediante rutas HTTP.

Estado global:

text

- grafo: Grafo
- sucursales: dict[str, Sucursal]
- active_transfers: dict (transferencias activas con hilos)
- logger: Logger

Rutas principales:

Ruta	Método	Propósito
------	--------	-----------

/	GET	Página principal
/cargar_datos	POST	Carga archivos CSV
/sucursal/<sid>	GET	Ver catálogo de sucursal
/sucursal/<sid>/agregar	POST	Agregar producto
/sucursal/<sid>/eliminar/<codigo>	POST	Eliminar producto
/sucursal/<sid>/rollback	POST	Deshacer última eliminación
/sucursal/<sid>/buscar	GET	Búsqueda avanzada
/sucursal/<sid>/encolar/<codigo>	POST	Encolar producto para despacho
/sucursal/<sid>/despachar	POST	Despachar producto
/sucursal/<sid>/procesar_ingreso	POST	Procesar cola de ingreso
/rutas	GET	Visualizar red de sucursales
/trasladar	POST	Iniciar transferencia de producto
/api/transferencias	GET	Estado de transferencias (AJAX)
/estructura/<sid>/<tipo>	GET	Visualizar estructura específica
/benchmark/<sid>	GET	Ver resultados de benchmark
/logs	GET	Ver archivo de logs

Tabla Resumen de Complejidades (Fase 2)

TAD	Operación	Complejidad
Producto	Constructor / Getters	O(1)
ListaEnlazada	insertar_al_final / insertar_ordenado	O(n)

ListaEnlazada	buscar / eliminar por código	$O(n)$
ArbolAVL	insertar / buscar / eliminar	$O(\log n)$
ArbolB	insertar / buscar / eliminar	$O(\log n)$
ArbolB	buscar_por_rango	$O(\log n + k)$
ArbolBPlus	insertar / buscar / eliminar	$O(\log n)$
ArbolBPlus	buscar_por_categoria	$O(\log n + k)$
TablaHash	insertar / buscar / eliminar	$O(1)$ promedio
Pila	push / pop	$O(1)$
Cola	enqueue / dequeue	$O(1)^*$
Grafo - Dijkstra	ruta mínima (una consulta)	$O((V+E) \log V)$
Grafo - Floyd	precalcular todas las rutas	$O(V^3)$
Grafo - Floyd	consulta (precalculado)	$O(1)$
Catalogo	agregar_producto (atómico)	$O(\log n)$ promedio
Catalogo	buscar_por_codigo	$O(1)$ promedio
Catalogo	buscar_por_nombre	$O(\log n)$
Catalogo	buscar_por_categoria	$O(\log n + k)$
Catalogo	buscar_por_rango	$O(\log n + k)$
CSVLoader	cargar_archivo	$O(n)$
ReportesGraphviz	generación de estructuras	$O(n)$
Benchmark	iniciar / finalizar	$O(1)$

Nota: k = número de resultados en la búsqueda

Estructuras de Datos Implementadas (Resumen Fase 2)

Estructura	Propósito	Clave de ordenación
-------------------	------------------	----------------------------

Lista Enlazada	Almacenamiento básico + comparativa	(ninguna / nombre)
Árbol AVL	Búsqueda por nombre	nombre
Árbol B	Búsqueda por rango de fechas	fecha_vencimiento
Árbol B+	Búsqueda por categoría	categoría
Tabla Hash	Búsqueda por código de barras	codigo_barras
Pila	Rollback (deshacer eliminaciones)	N/A
Cola	Simulación de flujo de productos	N/A (FIFO)
Grafo	Red de sucursales y rutas óptimas	tiempo / costo

Análisis de Complejidad (Big-O)

21.1. Conceptos Fundamentales

Para todas las estructuras, definimos:

n = número de productos almacenados

k = número de resultados en una búsqueda (rango o categoría)

V = número de vértices (sucursales) en el grafo

E = número de aristas (conexiones) en el grafo

21.2. Lista Enlazada

Operación	Complejidad	Justificación
Constructor ListaEnlazada()	$O(1)$	Solo inicializa head = None y size = 0
insertar_al_final(valor)	$O(n)$	Recorre toda la lista hasta

		encontrar el último nodo (n pasos)
insertar_ordenado_por_nombre(producto)	O(n)	Recorre comparando nombres hasta encontrar la posición de inserción
buscar_por_codigo(codigo)	O(n)	En el peor caso, el producto está al final o no existe (n comparaciones)
eliminar_por_codigo(codigo)	O(n)	Primero busca el nodo (O(n)) y luego elimina (O(1))
get_size()	O(1)	Solo retorna el contador almacenado

Análisis: La lista enlazada es la estructura más simple pero también la más lenta para búsquedas. Con $n=1000$, puede requerir hasta 1000 comparaciones.

21.3. Árbol AVL

Operación	Complejidad	Justificación
Constructor ArbolAVL()	O(1)	Solo inicializa raiz = None
insertar(producto)	O(log n)	Recorre desde la raíz hasta una hoja (altura $\approx 1.44 \log_2 n$) + rotaciones O(1)
buscar(nombre)	O(log n)	Recorre la altura del árbol, haciendo 1 comparación por nivel
eliminar(nombre)	O(log n)	Busca el nodo (log n) + rebalanceo hacia arriba (log n)
_rotacion_LL()	O(1)	Solo cambia 3 punteros, sin recorridos
_rotacion_DD()	O(1)	Solo cambia 3 punteros
_rotacion_ID()	O(1)	Solo cambia 4 punteros
_rotacion_DI()	O(1)	Solo cambia 4 punteros
_balancear(nodo)	O(1)	Revisa FE y aplica rotación si es necesario
_altura(nodo)	O(n)	Recorre todo el subárbol (usado internamente, no en operaciones)

principales)

Demostración de altura: En un AVL con n nodos, la altura $h \leq 1.44 * \log_2(n+2)$. Por lo tanto, el número de comparaciones nunca supera este valor.

Ejemplo: Con $n = 1000$, altura máxima $\approx 1.44 * \log_2(1002) \approx 1.44 * 10 \approx 14$ comparaciones.

21.4. Árbol B (orden $d=2$)

Operación	Complejidad	Justificación
Constructor ArbolB()	$O(1)$	Solo inicializa raiz = None
insertar(producto)	$O(\log n)$	Recorre desde raíz hasta hoja; en cada nodo, búsqueda lineal $O(d) = O(1)$
buscar(fecha)	$O(\log n)$	Misma razón: altura limitada + búsqueda lineal en nodo
buscar_por_rango(desde, hasta)	$O(\log n + k)$	Baja hasta la primera llave del rango ($\log n$) + recorre k resultados
eliminar(fecha)	$O(\log n)$	Busca ($\log n$) + posibles préstamos/fusiones $O(d) = O(1)$
_split(padre, i, hijo_lleno)	$O(d) = O(1)$	Copia como máximo $d = 2$ elementos
_fusionar(nodo, idx)	$O(d) = O(1)$	Fusiona hasta $2d = 4$ llaves
_prestar_de_anterior/siguiente()	$O(d) = O(1)$	Mueve como máximo d elementos

Demostración de altura: En un árbol B de orden $d=2$, la altura máxima es $\log_2((n+1)/2) + 1$.

Ejemplo: Con $n = 1000$, altura $\leq \log_2(1001/2) + 1 \approx \log_2(500) + 1 \approx 9 + 1 = 10$ niveles.

21.5. Árbol B+ (orden $d=2$)

Operación	Complejidad	Justificación
Constructor ArbolBPlus()	$O(1)$	Solo inicializa raiz = None
insertar(producto)	$O(\log n)$	Igual que árbol B, pero con manejo especial de hojas

buscar(categoria)	$O(\log n)$	Baja hasta la hoja ($\log n$) y busca lineal en la hoja $O(d)$
buscar_por_categoria(categoria)	$O(\log n + k)$	Baja a primera hoja ($\log n$) + recorre hojas enlazadas (k)
eliminar(categoria, codigo)	$O(\log n)$	Busca ($\log n$) + préstamo/fusión $O(1)$
_split(padre, i, hijo_lleno)	$O(d) = O(1)$	Similar a árbol B
_fusionar_hojas(padre, idx)	$O(d) = O(1)$	Fusiona hojas y actualiza puntero siguiente

Ventaja clave: Para buscar_por_categoria(), no es necesario volver a bajar desde la raíz para cada producto. Se recorre linealmente usando siguiente.

Ejemplo: Buscar "Lácteos" con $k=87$ productos: 10 niveles de descenso + 87 recorridos de hoja = ~ 97 operaciones vs $87 \times 10 = 870$ en un árbol B normal.

21.6. Tabla Hash

Operación	Complejidad	Justificación
Constructor TablaHash()	$O(\text{TAMANIO}) = O(1)$	101 cubetas es constante
_funcion_hash(llave)	$O(L) \approx O(1)$	L es longitud del código (ej. 13 caracteres) $\rightarrow$ constante
insertar(producto)	$O(1)$ promedio	Calcula hash $O(1)$ + inserta en lista $O(1)$ (al final, pero lista corta)
buscar(codigo)	$O(1)$ promedio	Hash + recorrido en lista de la cubeta (longitud promedio $\approx n/101$)
eliminar(codigo)	$O(1)$ promedio	Hash + recorrido en lista de la cubeta

Peor caso: $O(n)$ cuando todos los productos caen en la misma cubeta (función hash deficiente o mala distribución). Con djb2 esto es extremadamente improbable.

Factor de carga: $\alpha = n / 101$. Con $n=1000$, $\alpha \approx 9.9$. El tiempo de búsqueda es proporcional a α .

21.7. Pila (para Rollback)

Operación	Complejidad	Justificación
-----------	-------------	---------------

Constructor Pila()	O(1)	Solo inicializa items = []
push(item)	O(1)	Agrega al final de la lista (amortizado)
pop()	O(1)	Elimina del final de la lista
esta_vacia()	O(1)	Verifica len(items) == 0

Análisis: La pila es ideal para rollback porque siempre necesitamos deshacer la última eliminación (LIFO).

21.8. Cola

Operación	Complejidad	Justificación
Constructor Cola()	O(1)	Solo inicializa items = []
enqueue(item)	O(1)	Agrega al final de la lista (amortizado)
dequeue()	O(1)	Elimina del inicio (pop(0) en Python es O(n), pero conceptualmente O(1))
esta_vacia()	O(1)	Verifica len(items) == 0

Nota: En mi implementación con lista de Python, pop(0) es O(n). Para una implementación óptima se usaría collections.deque. Para este proyecto, con colas pequeñas, no afecta significativamente.

21.9. Grafo

Operación	Complejidad	Justificación
agregar_sucursal(nombre)	O(1)	Inserta en diccionario y lista de vértices
agregar_camino(origen, destino, tiempo, costo)	O(1)	Agrega arista en ambos sentidos (grafo no dirigido)

dijkstra(inicio, fin, criterio)	$O((V+E) \log V)$	Cada vértice se extrae del heap una vez ($\log V$) y cada arista se relaja una vez
floyd()	$O(V^3)$	Tres ciclos anidados de V iteraciones cada uno
obtener_ruta()	$O((V+E) \log V)$	Ejecuta Dijkstra internamente
obtener_ruta_floyd()	$O(V^3)$ precálculo + $O(1)$ consulta	Precálculo costoso una vez, luego consultas inmediatas

Ejemplo con $V=5$ sucursales, $E=8$ aristas:

Dijkstra: $\approx (5+8) \log_2 5 \approx 13 \times 2.3 \approx 30$ operaciones

Floyd-Warshall: $5^3 = 125$ operaciones de precálculo

21.10. Tabla Resumen de Complejidades (TADs)

TAD	Operación	Mejor Caso	Caso Promedio	Peor Caso
Producto	Constructor	$O(1)$	$O(1)$	$O(1)$
ListaEnlazada	insertar_al_final	$O(1)^*$	$O(n)$	$O(n)$
	buscar_por_codigo	$O(1)$	$O(n)$	$O(n)$
	eliminar_por_codigo	$O(1)$	$O(n)$	$O(n)$
ArbolAVL	insertar	$O(\log n)$	$O(\log n)$	$O(\log n)$
	buscar	$O(\log n)$	$O(\log n)$	$O(\log n)$
	eliminar	$O(\log n)$	$O(\log n)$	$O(\log n)$
ArbolB (d=2)	insertar	$O(\log n)$	$O(\log n)$	$O(\log n)$
	buscar	$O(\log n)$	$O(\log n)$	$O(\log n)$
	buscar_por_rango	$O(\log n + k)$	$O(\log n + k)$	$O(\log n + k)$

	eliminar	$O(\log n)$	$O(\log n)$	$O(\log n)$
ArbolBPlus (d=2)	insertar	$O(\log n)$	$O(\log n)$	$O(\log n)$
	buscar_por_categoria	$O(\log n + k)$	$O(\log n + k)$	$O(\log n + k)$
	eliminar	$O(\log n)$	$O(\log n)$	$O(\log n)$
TablaHash	insertar	$O(1)$	$O(1)$	$O(n)$
	buscar	$O(1)$	$O(1)$	$O(n)$
	eliminar	$O(1)$	$O(1)$	$O(n)$
Pila	push / pop	$O(1)$	$O(1)$	$O(1)$
Cola	enqueue / dequeue	$O(1)$	$O(1)$	$O(1)$
Grafo	Dijkstra (una ruta)	$O((V+E) \log V)$	$O((V+E) \log V)$	$O((V+E) \log V)$
	Floyd-Warshall (precálculo)	$O(V^3)$	$O(V^3)$	$O(V^3)$
	Floyd-Warshall (consulta)	$O(1)$	$O(1)$	$O(1)$

**insertar_al_final con puntero a cola sería $O(1)$; mi implementación recorre hasta el final $\rightarrow O(n)$ *